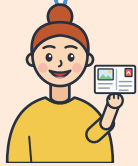


3. HTTPS: A SEALED LETTER, NOT A POSTCARD

ANALOGY: POSTCARD OR SEALED LETTER?

1 HTTP: A POSTCARD

My password is tulip7.



Anna

postcard



Anyone on the way reads it, can even change it

postcard, maybe altered



Shop

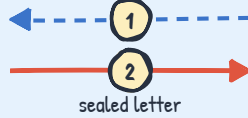
2 HTTPS: ID CHECK FIRST, THEN THE LETTER

Show me your ID first.



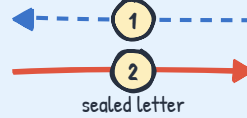
Anna

passport



Anyone on the way sees only the envelope

passport



Here is my passport.



Shop

HTTPS is HTTP inside TLS, on port 443 instead of 80. Same letter; TLS adds the seal and the ID check.

THREE PROMISES OF THE SEAL



CONFIDENTIALITY

Nobody on the way can read the letter.



INTEGRITY

A broken seal is noticed. Changed bytes fail the check.



AUTHENTICATION

The server shows its certificate, its passport.

WHY TRUST THE PASSPORT?



Root CA (already trusted)

signs



Intermediate CA

signs



example.com certificate

Your browser or OS trusts a short list of root CAs. No chain of signatures up to one of them: a warning page.

TLS 1.3 HANDSHAKE: ONE ROUND TRIP

1. CLIENTHELLO

Browser: TLS 1.3, please. Here is my key share.



2. SERVERHELLO

Server's key share. Both derive the same fresh secret (Diffie-Hellman).



3. CERTIFICATE

Sent sealed, plus a signature. Its key only signs, never encrypts.



4. SEALED DATA

Browser checks chain, name, signature. Then HTTP flows encrypted.



WHAT A WATCHER STILL SEES



HTTP = POSTCARD



HTTPS = LETTER

THE LETTER

readable

sealed: path, cookies

RECIPIENT NAME

visible (Host)

usually visible (SNI)

THE ENVELOPE

visible

visible: IPs, timing, size

LOOK AT THE SEAL YOURSELF

```
$ curl -v https://example.com
prints the TLS version and certificate details

$ openssl s_client \
  -connect example.com:443
lists the certificate chain the server sends
```



REMEMBER: HTTPS seals the letter and checks the ID. The envelope is still visible.